

Real-Time Food Delivery Analytics Platform

Finalized Design — A Data Engineering Practice Lab Built Around Reusable Patterns

This is the finalized, cleaned-up design. Rather than a list of tools, it is organized around the DESIGN PATTERNS a Data Engineer is actually hired to know — each pattern is stated as a problem, a solution, and where it lives in this project. Build the project to learn the patterns; talk about the patterns to pass the interview.

How To Read This Document

- **Start here:** Section 1 gives the one-page mental model — read this first and keep it in your head.
- **Architecture:** Section 2 is the layered architecture, drawn cleanly so each layer has ONE job.
- **Patterns:** Section 3 is the heart of the document — 12 named design patterns, each in a Problem / Solution / In-this-project format. This is your interview vocabulary.
- **Build:** Sections 4-6 are the build plan (repo layout, local Docker, Kubernetes) once you understand the design.
- **Practice:** Section 7 is the 'Break the Pipeline' lab; Section 8 maps everything to resume bullets.

1. The One-Page Mental Model

Every data platform, no matter the company, is the same five jobs in a line. Learn to see this shape and every architecture becomes readable:

GENERATE	->	MOVE	->	STORE (raw)	->	TRANSFORM	->	SERVE
events (orders, payments, drivers)		stream /queue		immutable landing zone (never edit)		clean + model into facts/dims		dashboards & queries

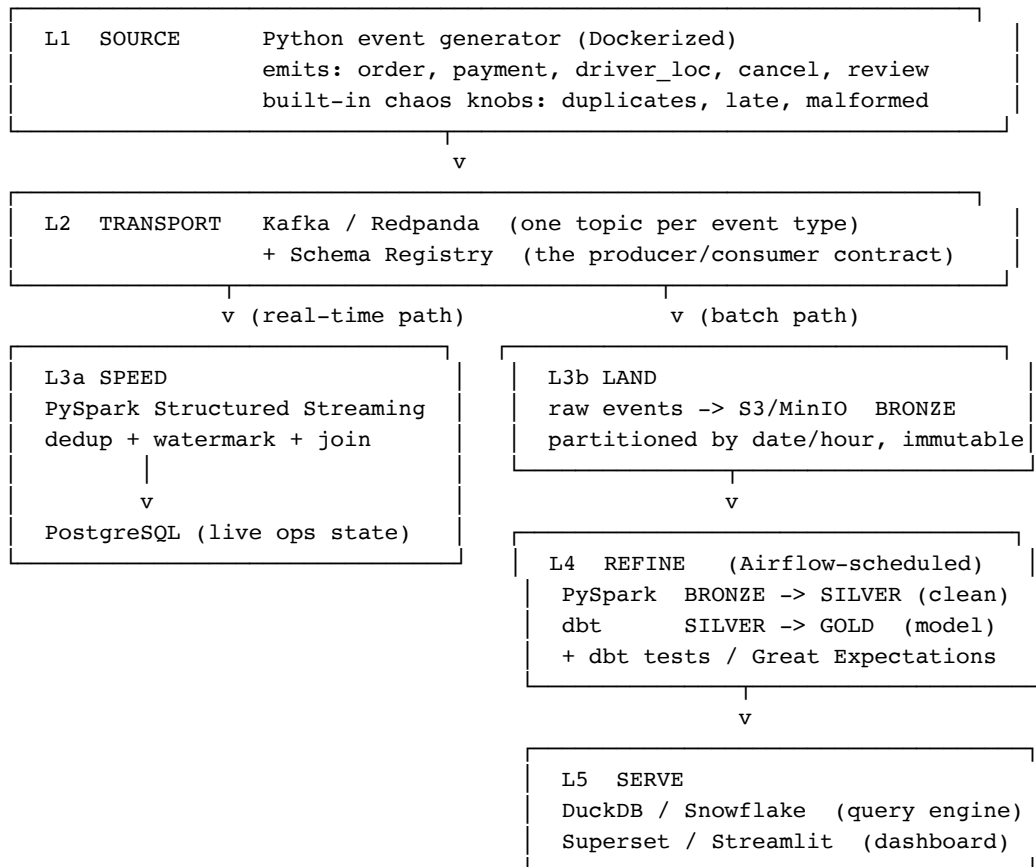
Two independent speeds run over this same shape:

- **Speed layer (real-time):** moving data continuously, second-by-second, for live operational views (the streaming path).
- **Batch layer (scheduled):** moving data in scheduled chunks for correct, complete, historical analytics (the batch path).

Everything else in this document — Kafka, PySpark, Airflow, dbt, the medallion lake, Kubernetes — is just a concrete tool filling one box in that line. When an interviewer describes a system, mentally drop each piece into GENERATE / MOVE / STORE / TRANSFORM / SERVE and you will never feel lost.

2. Layered Architecture (Each Layer Has One Job)

The finalized design is drawn as clean horizontal layers. Data only ever flows downward; each layer depends only on the one directly above it, which is what keeps the system readable and debuggable.



CROSS-CUTTING (wrap every layer):

- Orchestration — Airflow (KubernetesExecutor) schedules L3b->L5
- Packaging — every box is a Docker image
- Runtime — docker-compose (dev) -> Kubernetes (prod-like)
- CI/CD — GitHub + GitHub Actions build/test/deploy
- Observability — logs, Airflow UI, data-quality alerts

3. The 12 Core Design Patterns (Your Interview Vocabulary)

This is the most important section. Each pattern below is something a Data Engineer is expected to name and explain. Learn the PATTERN, not just the tool — tools change between companies, patterns do not.

Group A — Storage & Layout Patterns

3.1 Medallion Architecture (Bronze / Silver / Gold)

Problem it solves: Where do you put data at each stage of cleanliness, and how do you recover when transformation logic has a bug?

How it works: Three named layers: Bronze is raw and immutable (never edited — your replayable source of truth); Silver is cleaned, deduplicated, schema-conformed; Gold is business-modeled facts and dimensions. Because Bronze is immutable, you can always rebuild Silver and Gold from scratch when logic changes, without re-ingesting from source.

In this project: S3/MinIO folders bronze/, silver/, gold/. PySpark builds Silver; dbt builds Gold.

3.2 Hive-Style Partitioning

Problem it solves: How do you avoid scanning an entire table when a query only needs one day of data?

How it works: Encode partition columns into the storage path (year=2024/month=01/day=15/). The query engine reads only the matching folders (partition pruning) instead of the whole table — the single biggest lever for both speed and cost.

In this project: All lake layers partitioned by event_date/event_hour; queries filter on those columns so pruning kicks in.

3.3 Columnar File Format (Parquet)

Problem it solves: How do you make analytical queries that touch a few columns across millions of rows fast?

How it works: Store data column-by-column (Parquet/ORC) instead of row-by-row. A query selecting 3 of 30 columns reads only those 3 columns' data plus gets far better compression — versus a row format (CSV/JSON) that must read every byte.

In this project: Everything past Bronze is written as Snappy-compressed Parquet.

Group B — Streaming & Correctness Patterns

3.4 Idempotency (Safe Re-Runs)

Problem it solves: A job fails halfway and you must restart it — how do you avoid creating duplicate data?

How it works: Design every write so running it twice produces the same result as running it once: overwrite-by-partition instead of blind append, and UPSERT (INSERT ... ON CONFLICT) keyed on a natural business key instead of plain INSERT. This is what makes retries safe rather than dangerous.

In this project: Batch jobs overwrite the target partition; Postgres writes use UPSERT on `order_id`; Spark uses checkpointing.

3.5 Exactly-Once via Checkpointing

Problem it solves: How does a streaming job resume after a crash without reprocessing or dropping events?

How it works: The stream processor persists its progress (which source offsets it has committed) to durable storage. On restart it reads the checkpoint and resumes precisely where it stopped — no gaps, no double-counting.

In this project: PySpark Structured Streaming with `checkpointLocation` on MinIO; Kafka offsets committed per micro-batch.

3.6 Watermarking for Late / Out-of-Order Data

Problem it solves: Payment events sometimes arrive before their order events — how do you join them correctly without waiting forever?

How it works: Process on EVENT time (when it happened), not processing time (when it arrived). A watermark ('accept events up to 10 minutes late') tells the engine how long to keep a window open before finalizing it. Events within the bound are correctly placed; events past it go to a dead-letter path instead of silently corrupting results.

In this project: `withWatermark('event_time','10 minutes')` on both streams; stream-stream join with a time constraint.

3.7 Deduplication on a Natural Key

Problem it solves: A restaurant's app sends the same order twice — how do you keep exactly one copy?

How it works: Deduplicate on a true business key (`order_id`), never on a system-generated ingestion timestamp or row id (two copies of the same event have different ingestion times but the same `order_id`). Do it in two passes: a best-effort pass in streaming, and a thorough pass in the Silver batch job for duplicates that arrive later.

In this project: `dropDuplicates(['order_id'])` in streaming; a wider windowed dedup in the Bronze->Silver PySpark job.

Group C — Modeling & Evolution Patterns

3.8 Dimensional Modeling (Star Schema)

Problem it solves: Analysts want 'delivery time by city, restaurant, and hour' — how do you make that query fast and intuitive?

How it works: Model Gold as a central FACT table (one row per delivery, with numeric measures + foreign keys) surrounded by DIMENSION tables (city, restaurant, driver, date). Optionally pre-aggregate the most common rollup into a small summary table the dashboard hits directly.

In this project: `fact_deliveries` + `dim_city` / `dim_restaurant` / `dim_driver` / `dim_date`, built by dbt.

3.9 Slowly Changing Dimension Type 2 (History Preservation)

Problem it solves: A restaurant changes its name or a driver moves cities — how do you keep historical reports accurate?

How it works: Instead of overwriting the dimension row, insert a NEW row and expire the old one (`effective_start_date`, `effective_end_date`, `is_current`, and a surrogate key). Old facts point at the old row; new facts at the new one — so 'what was true at the time' is preserved.

In this project: `dim_restaurant` / `dim_driver` modeled as SCD Type 2 in dbt with snapshot logic.

3.10 Schema Evolution via a Contract

Problem it solves: A field renames from `price` to `order_price` — how do you not break every downstream consumer overnight?

How it works: Treat schema as a versioned contract: every event carries a `schema_version`; a Schema Registry enforces backward-compatible changes (a producer literally cannot publish a breaking change); the Silver job aliases old field names to one canonical name during a transition window.

In this project: Redpanda/Confluent Schema Registry in BACKWARD compatibility mode; schema_version on every event.

Group D — Quality & Operations Patterns

3.11 Data Quality as a Pipeline Gate

Problem it solves: How do you stop bad data from silently reaching a dashboard?

How it works: Run automated checks as an explicit step that FAILS the pipeline (and alerts) when violated — not something you eyeball afterward. Two tiers: structural tests (unique, not_null, foreign-key relationships) and statistical/business-rule tests (delivery_time between 5 and 120 minutes; volume not down >50% day-over-day).

In this project: dbt tests for structural checks; Great Expectations for statistical checks, both as Airflow tasks that block on failure.

3.12 Orchestration as Idempotent DAGs

Problem it solves: How do you run a multi-step pipeline reliably with dependencies, retries, and failure alerts?

How it works: Model the pipeline as a Directed Acyclic Graph where each task is independently idempotent, has retry/backoff configured, and alerts on failure. Because every task is safe to re-run (Pattern 3.4), a retry after a mid-DAG failure never duplicates data.

In this project: Airflow DAG: sense new data -> PySpark Silver -> dbt Gold -> data-quality gate -> refresh dashboard, each with retries.

4. Repository Structure (Monorepo)

One repository, one folder per component, so the whole system is readable at a glance and CI can build each piece independently.

```
food-delivery-analytics-platform/
├── README.md                # the Section 1 mental model + Section 2 diagram +
├── setup                    #
├── docker-compose.yml       # full local stack, one command to run everything
├── .github/workflows/       # ci.yml (lint+test)  cd.yml (build/push/deploy)
├──
├── L1_generator/            # event generator + chaos config
├── L3a_streaming/           # PySpark Structured Streaming job
├── L3b_L4_batch/           # PySpark bronze->silver job
├── dbt_project/             # silver->gold models + tests (models/silver, models/
gold)
```

```

├─ great_expectations/      # statistical data-quality suites
├─ airflow/dags/             # the orchestration DAG
├─ dashboard/                # Streamlit / Superset app
├─ control_panel/            # "Break the Pipeline" fault-injection service
├─
└─ k8s/                       # StatefulSets, Helm values, Spark-operator job specs

```

- **Readability tip:** Folder names are prefixed with their architecture layer (L1, L3a...) so the repo layout mirrors the diagram — anyone opening the repo can map code to design immediately.

5. Local Development with Docker Compose

The whole point of docker-compose here is that ONE command brings up the entire platform reproducibly. Build it incrementally so you never debug five integrations at once.

1. **Milestone 1:** generator -> Redpanda -> a PySpark consumer that just prints events. Prove data flows.
 2. **Milestone 2:** add PostgreSQL + MinIO; land raw events to Bronze and upsert live state to Postgres.
 3. **Milestone 3:** add Airflow + dbt; build Silver and Gold on a schedule.
 4. **Milestone 4:** add Great Expectations + the dashboard; close the loop end to end.
 5. **Milestone 5:** add the control panel and start breaking things on purpose.
- **Two rules that save hours:** Use named volumes for postgres-data and minio-data so state survives restarts; put everything on one Docker network so services reach each other by name (redpanda:9092, not localhost).

6. Kubernetes Deployment (Production-Like)

Once docker-compose works end to end, migrating to Kubernetes is the step that makes this genuinely interview-grade — it is how real data platforms run, and it is a concrete talking point most candidates lack.

- **StatefulSets** for stateful services (Postgres, Redpanda, MinIO) — stable identity + durable PersistentVolumeClaims.
- **Airflow** deployed via its official Helm chart with the KubernetesExecutor, so each DAG task runs as its own pod (the same pattern companies run on EKS).
- **PySpark jobs** submitted through the Spark-on-Kubernetes operator, which spins up driver/executor pods per job — directly analogous to EMR-on-EKS.
- **CronJob** for the scheduled small-file compaction job; ConfigMaps for config; Secrets for credentials (never in the image or Git).

- **An Ingress** exposes the dashboard, matching how a real service is reached rather than port-forwarding into a pod.

7. The "Break the Pipeline" Lab (Active Practice)

This is what converts a passive build into interview-ready instinct. A small FastAPI + Streamlit control service injects faults into the running system; you diagnose and repair. Each fault maps to a pattern above.

FAULT INJECTED	PATTERN YOU PRACTICE
raise <code>duplicate_rate</code> on generator	→ 3.7 Deduplication
raise <code>out_of_order_rate</code>	→ 3.6 Watermarking / late data
kill the streaming pod mid-run	→ 3.5 Checkpointing / recovery
kill Airflow mid-DAG	→ 3.4 Idempotent re-run
publish an event with a renamed field	→ 3.10 Schema evolution
fire a customer-deletion request	→ privacy-safe deletion (below)
point dashboard at unpartitioned data	→ 3.2 Partitioning / query tuning

The loop every time: inject -> observe (dashboard/logs/Airflow) -> diagnose the layer -> fix -> confirm recovery. Do this repeatedly and scenario interview questions stop being scary.

Privacy-Safe Deletion (the one fault with no single pattern)

Parquet files are immutable, so deletion is not a DELETE statement. The approach: locate the affected partitions/files, read each, filter out the customer's rows, and rewrite the file (a targeted compaction); apply it in every layer independently (Postgres DELETE, plus lake rewrites); and log the deletion for audit. The production-grade upgrade — worth naming in an interview — is adopting a table format with native DELETE/MERGE (Apache Iceberg, Delta Lake, or Hudi) instead of hand-rolling file rewrites.

8. Turning This Into Interview Currency

Lead with patterns and trade-offs, not tool names — that is the difference between 'I followed a tutorial' and 'I understand why it is built this way.'

- **Bullet:** "Designed a medallion-architecture lakehouse (bronze/silver/gold on S3-compatible storage) with idempotent, checkpointed streaming ingestion, handling duplicate and out-of-order events via natural-key deduplication and event-time watermarking."
- **Bullet:** "Modeled a star schema (fact_deliveries + SCD Type 2 dimensions) with dbt, gated by automated dbt tests and Great Expectations quality checks wired into an Airflow DAG."
- **Bullet:** "Containerized every component and deployed on Kubernetes with the Airflow KubernetesExecutor and Spark-on-Kubernetes, with a GitHub Actions CI/CD pipeline."

- **Bullet:** "Built a fault-injection tool to practice production incident response — schema drift, pipeline restarts, and privacy-driven deletion."

Pair this with your AWS Data Engineer 30-Day prep: Redpanda≈Kinesis/MSK, PySpark≈Glue/EMR, Airflow≈MWAA, MinIO≈S3, Kubernetes≈EKS — the patterns are identical, only the managed-service names change.